Savitribai Phule Pune University

A

Lab. Practice-V

Mini-Project

Report on

"Multiplayer Carrom Game"

Submitted in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF ENGINEERING IN INFORMATION TECHNOLOGY

[B.E. Information Technology]

By

Om Balasaheb Taskar (Roll no)

Umesh Kailas Shingare (Roll no)

Saurav Anil Sultane (Roll no)

Kalpesh Subhash Wagh (Roll no)

Om Vishnu Autade (Roll no)

At

BHUJBAL KNOWLEDGE CITY Mumbai Educational Trust | Institute of Engineering | +91 9881100099, 7264063555 Adgaon, Nashik-422003 | FOLLOW US | ISO 9001:2015 CERTIFIED

Department of Information Technology

MET's INSTITUTE OF ENGINEERING

Aadgaon, Nashik – 422213.

Academic Year 2025 - 2026

MET's INSTITUTE OF ENGINEERING

Aadgaon, Nashik – 422213.

Department of Information Technology

BE IT (2025-26)

Certificate

This is to certify that, the Lab. Practice-V Mini Project report **"Multiplayer Carrom Game"** submitted by **Om Taskar, Umesh Shingare, Saurav Sultane, Kalpesh Wagh, Om Autade** for partial fulfillment of the requirement for the award of the **Bachelor of Engineering** in **Information Technology** at **MET's Institute of Engineering, Nashik** as laid down by the **Savitribai Phule Pune University**. This is a record of the work carried out under my supervision and guidance during academic year 2025 - 2026.

Place: - Nashik.

Date: - / / 2026.

Mr. Ratan D. Deokar
Mini Project Guide

Dr. N. R. Kale
HOD IT

INDEX

1. Abstract
2. Introduction
3. Importance of Project topic
4. Methodology/ Flow of your system
5. List of Table, Entities, attributes
6. ER Diagram of Project
7. All Tables with inserted values
8. Software and Hardware Requirements
9. Conclusion

1.ABSTRACT

This project presents the comprehensive design and implementation of a Distributed Application for Interactive Multiplayer Games, focusing specifically on the development of a real-time digital Carrom board. The core objective of this application is to demonstrate the fundamental principles of distributed systems, including low-latency data synchronization, robust state management, and concurrent user interaction across a network. The system utilizes a robust client-server architecture to enable multiple users to play a highly responsive, physics-based game concurrently, irrespective of their physical locations.

By employing an authoritative server model, the application shifts complex computational workloads—specifically the execution of the rigid body physics engine (Matter.js)—to the centralized Node.js backend. This architectural decision ensures that game states are deterministically calculated and perfectly synchronized across all connected clients at a smooth 60 frames per second via Socket.io. Crucially, this server-side validation completely eliminates the possibility of client-side manipulation, thereby preventing cheating and ensuring a fair, competitive environment. The frontend, built utilizing HTML5 Canvas and Vanilla JavaScript, acts as a lightweight rendering terminal that captures user inputs and visualizes the synchronized physics data in a premium, responsive user interface. Ultimately, this project serves as a practical demonstration of how modern web technologies can be leveraged to build scalable, secure, and highly interactive distributed architectures.

2.INTRODUCTION

In contemporary software architecture, distributed applications play a pivotal role. These are sophisticated systems wherein individual software components are dispersed across multiple networked computers, yet they communicate and coordinate their actions seamlessly by passing messages to appear as a single, unified application to the end-user. This mini-project practically demonstrates these complex principles through the development of an interactive, real-time multiplayer Carrom game.

Unlike traditional turn-based digital board games, physical tabletop simulations like Carrom require extremely low-latency data transfer to accurately replicate the fast-paced, chaotic collisions of a striker hitting multiple coins across a frictionless board. To achieve this, the application departs from standard stateless HTTP protocols and instead leverages WebSockets to establish persistent, full-duplex communication channels between the players and the server.

The backend infrastructure is engineered using Node.js, combined with the Socket.io library, to manage numerous concurrent player connections efficiently without blocking the event loop. Furthermore, to maintain a single source of truth, a dedicated 2D rigid-body physics engine (Matter.js) is executed entirely on the server. The client-side application is deliberately designed as a "dumb terminal"—constructed purely with Vanilla JavaScript and the HTML5 Canvas API—whose sole responsibilities are to capture raw user input (mouse drag vectors) and continuously render the authoritative coordinate data broadcasted by the server. This strict separation of concerns highlights best practices in building secure, distributed, and highly interactive web applications.

3.IMPORTANCE OF PROJECT TOPIC

The study and practical implementation of distributed systems is arguably one of the most critical domains in modern software engineering. As applications scale globally, the ability to coordinate state across vast geographical distances becomes paramount. This specific project, a real-time multiplayer game, serves as an excellent sandbox to explore these complex concepts under extreme constraints—specifically, the demand for ultra-low latency. It highlights several highly important architectural concepts:

Overcoming the Stateless Web: Traditional HTTP requests are stateless and unidirectional, making them unsuitable for live interactions. This project demonstrates the transition to stateful, full-duplex communication using WebSockets, allowing the server to push updates to clients unprompted.

Authoritative Server Architecture: In distributed applications, deciding "who holds the truth" is a massive architectural decision. By centralizing the physics logic on the Node.js server, this project completely mitigates client-side manipulation (cheating), which is a standard and necessary practice in both the gaming industry and secure financial applications.

High-Frequency State Synchronization: Learning how to serialize, compress, and broadcast state changes continuously at 60 frames per second over a network without causing bandwidth bottlenecks or memory leaks is a highly transferable skill to domains like real-time financial trading dashboards or collaborative document editing.

-**Deterministic Simulation Integration:** Bridging mathematical physics engines (calculating rigid-body dynamics, restitution, and friction) with networked environments requires precise synchronization to ensure all nodes in the distributed system converge on the exact same state simultaneously.

4.METHODOLOGY / FLOW OF SYSTEM

To achieve a seamlessly synchronized experience, the system strictly adheres to a centralized Client-Server communication flow, operating in distinct, continuous phases:

1. **Handshake & Role Assignment Phase:**Upon a user accessing the web interface, the client initiates a WebSocket handshake with the server. The server tracks active connections in memory and dynamically assigns roles. The first two connections are granted authoritative "Player 1" and "Player 2" roles, granting them input privileges, while all subsequent connections are gracefully downgraded to "Spectators" who only receive read-only broadcast data.

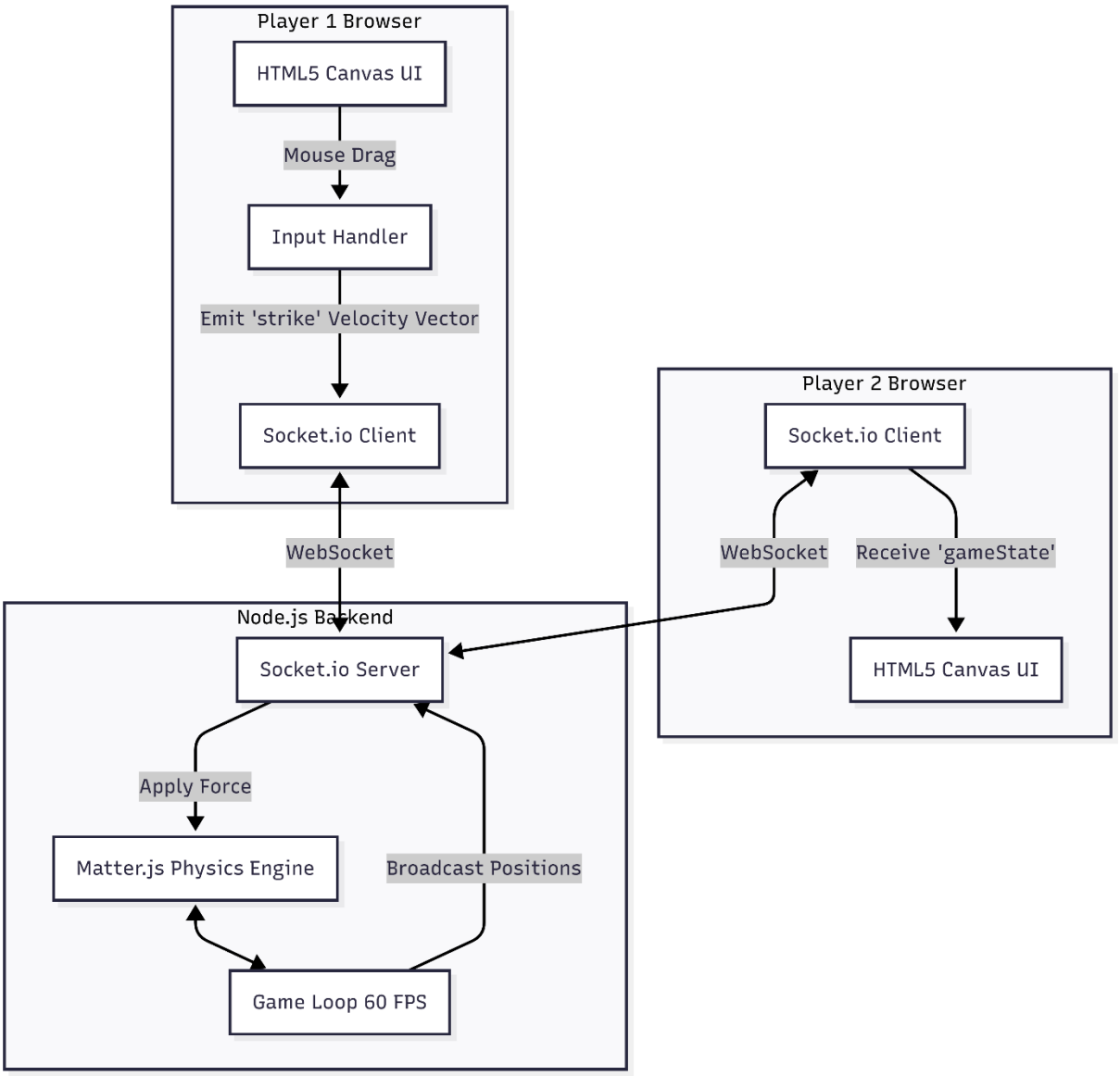
2.**The Input Capture Loop:** When the server determines it is a specific player's turn, the client-side JavaScript activates event listeners on the HTML5 Canvas. The user aims the striker via mouse drag. Crucially, the client does *not* calculate the outcome of the strike locally; it only calculates the raw directional velocity vector (vx, vy) and transmits this lightweight payload to the server via a `strike` event.

3.**The Simulation Loop (Server-Side):** Upon receiving the input vector, the Node.js server validates the request (ensuring the user is authorized to strike) and applies the force vector to the "Striker" body within the headless Matter.js physics world. The Matter.js engine calculates rigid body dynamics—including elastic collisions, air friction, and momentum transfer—updating the internal coordinates of all coins.

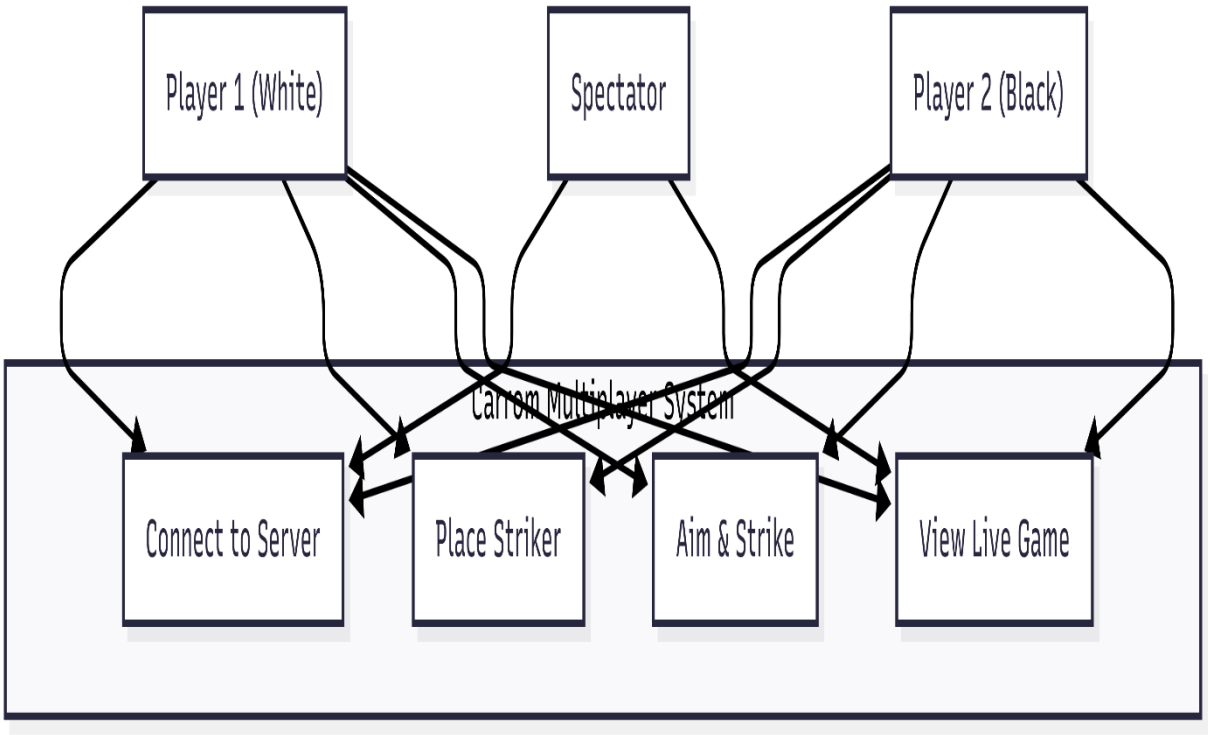
4.**The Broadcast Loop:** Utilizing a `setInterval` loop running at roughly 60 Hz (16.6ms intervals), the server iterates through all physical bodies in the Matter.js world. It constructs a highly optimized, minimized JSON payload containing only the necessary identifiers, X, and Y coordinates, and broadcasts this `gameState` event to every connected WebSocket client simultaneously.

5.**The Render Phase:** The client receives the broadcasted state payload. It immediately wipes the HTML5 Canvas clear and redraws the board, the pockets, the coins, and the striker precisely at the coordinates dictated by the server. Because this happens 60 times a second, the human eye perceives it as smooth, real-time motion.

SYSTEM ARCHITECTURE & DATA FLOW DIAGRAM



USE CASE DIAGRAM



5.LIST OF TABLES, ENTITIES, AND ATTRIBUTES

Table Name: USERS

Attributes:

1.user_id (Primary Key, INT)

2.username (VARCHAR)

3.total_score (INT)

4.join_date (DATETIME)

Table Name: MATCHES

Attributes:

1.match_id (Primary Key, INT)

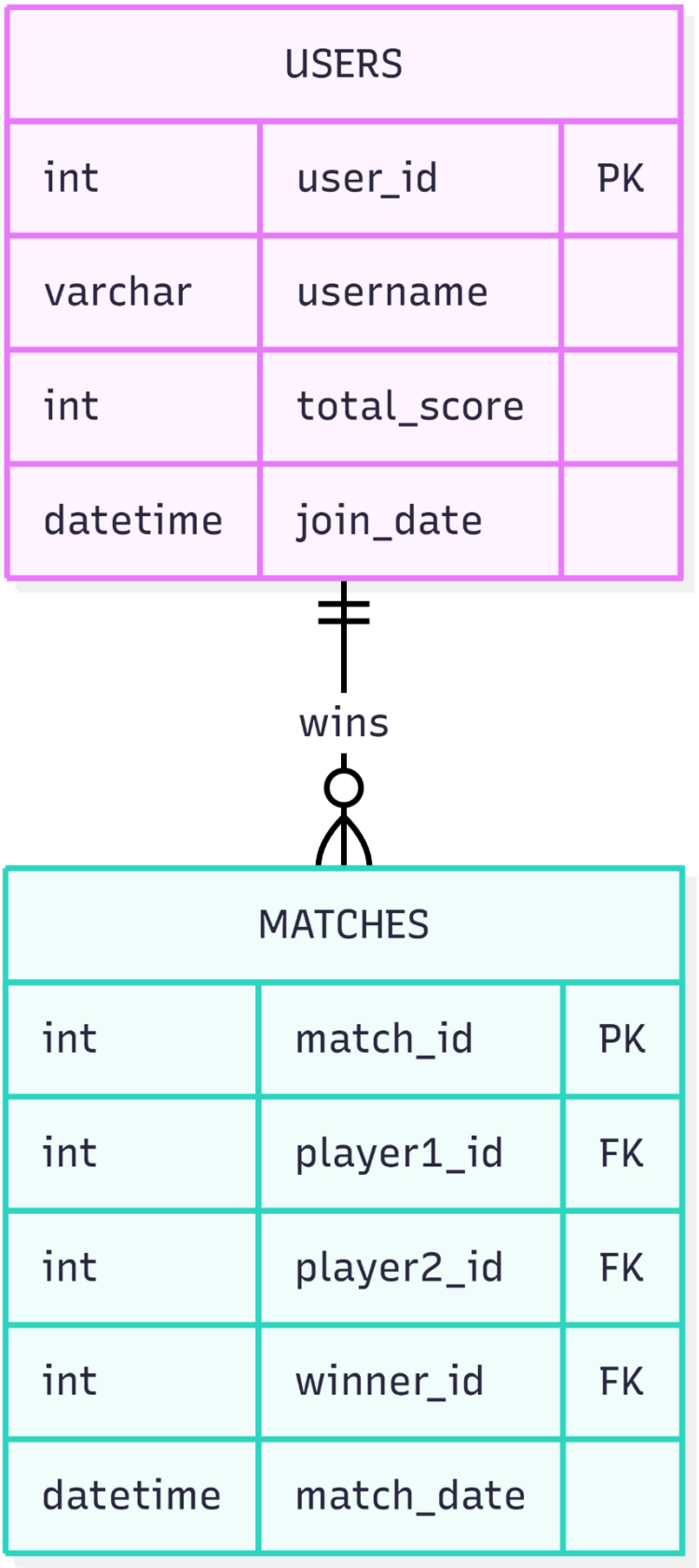
2. player1_id (Foreign Key → USERS.user_id, INT)

3. player2_id (Foreign Key → USERS.user_id, INT)

4. winner_id (Foreign Key → USERS.user_id, INT, Nullable)

5. match_date (DATETIME)

6.ER DIAGRAM OF PROJECT



7.ALL TABLES WITH INSERTED VALUES

User Table

user_id	username	total_score	join_date
101	PlayerOne_Pro	450	01-04-2026 10:00 AM
102	CarromKing	320	02-04-2026 02:30 PM
103	Guest_8829	0	25-04-2026 09:15 AM

Matches Table

Match_id	player1_id	player2_id	winner_id	match_date
5001	101	102	101	10-04-2026 06:00 PM
5002	102	103	102	25-04-2026 09:20 AM
5003	101	103	Null	25-04-2026 10:00 AM

8.SOFTWARE AND HARDWARE REQUIREMENTS:

1.Software Requirements

• **Backend Environment**:
Node.js (v16.0 or higher) is used due to its asynchronous and event-driven architecture, which efficiently handles multiple client connections and real-time communication.

• **Backend Libraries:**
- Express.js: Used to serve web pages and manage server-side routing.
- Socket.io: Enables real-time, bidirectional communication between client and server.
- Matter.js: A 2D physics engine used to simulate realistic game mechanics.

• **Frontend Technologies**:
HTML5, CSS3, and JavaScript are used to build the user interface. The HTML5 Canvas API is used for rendering smooth game animations.

• **Browser**:
Any modern web browser such as Google Chrome, Mozilla Firefox, Safari, or Microsoft Edge with JavaScript enabled.

2.Hardware Requirements

1. **Processor:** Intel i3 or higher
2. **Storage:** At least 500 MB free space
3. **Network**: Stable internet connection for multiplayer functionality
4. **Input Devices**: Keyboard and Mouse

9.CONCLUSION

This mini-project successfully fulfills the objectives of building a Distributed Application for Interactive Multiplayer Games. By utilizing Node.js and Socket.io for persistent WebSockets, the application achieves instantaneous, real-time communication between multiple clients.

The implementation of a completely server-authoritative physics engine using Matter.js ensures a fair, mathematically deterministic, and synchronized gameplay experience that inherently prevents client-side cheating. The resulting Carrom game is fully functional, highly responsive, and effectively demonstrates the core principles of scalable distributed state management, providing a robust architecture for future expansions.